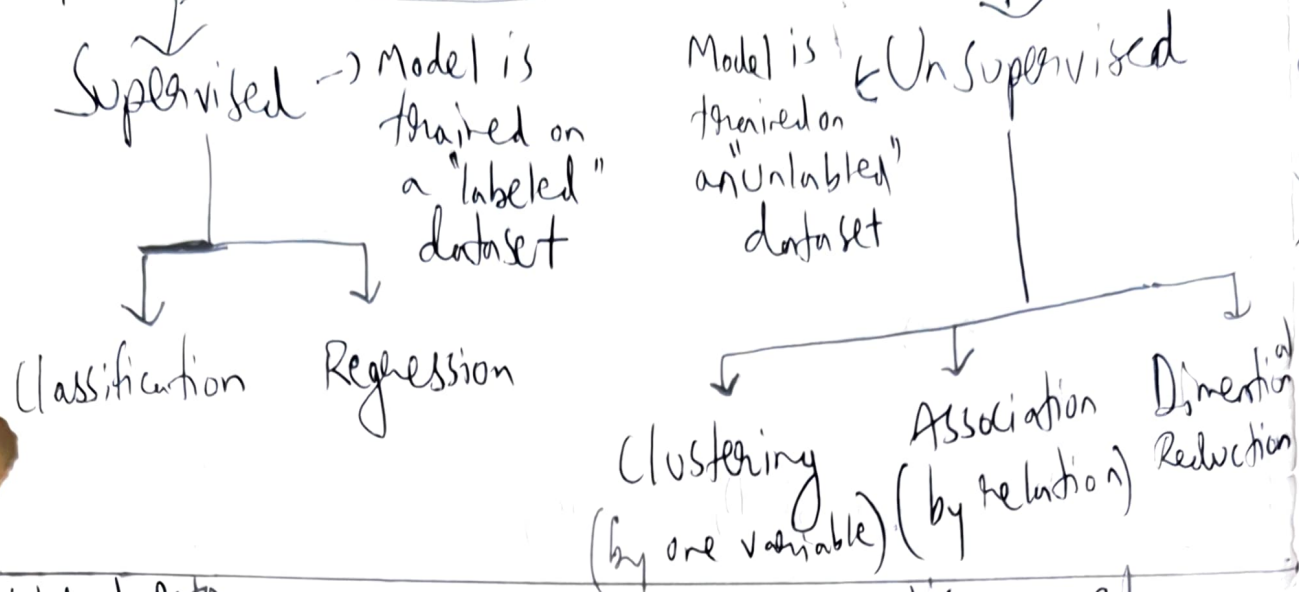


need human intervention $\downarrow$ Learning Neural Networks $\rightarrow$ Universal fn Approximators



Not Real World Data $\rightarrow$ Eg: 1 layer, 1 neuron, Linear regression, Logistic regression, etc..

Linear Model : $y = w_1x_1 + \dots + b$

Non-Linear Model : $y = w_1x_1^2 + \sin(w_2x_2)$

Real world Data $\rightarrow$ Eg: Decision Tree, ANN, RNN, CNN etc., and better

Feature Scaling $\rightarrow$ mainly used so as to get a faster loss calculation
 () one features should not dominate other

Normalization $\rightarrow$ feature values go to 0 & 1

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

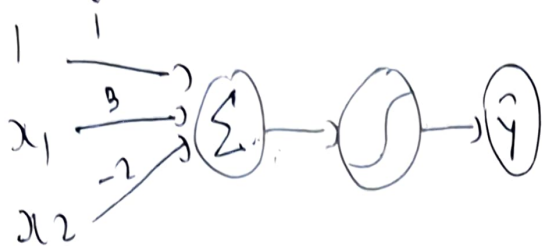
 When data is in specific range Here min, max, mean are important

Standardization $\rightarrow$ data converted into mean of 0 and std deviation of 1

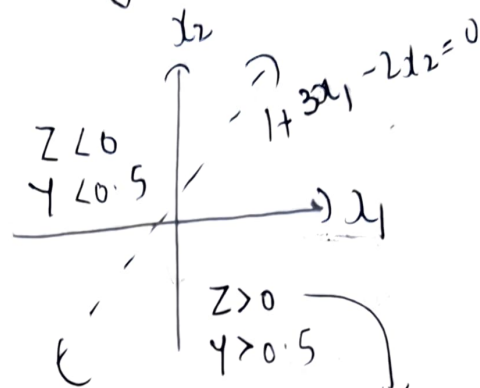
$$Z = \frac{x - \bar{x}}{\sigma}$$

 Here mean is important

For single Perceptron,



$$\hat{y} = g(1 + 3x_1 - 2x_2)$$



Learning Rates can be set

for example: Using SGD, Adam,

Adadelta, Adagrad, RMS Prop

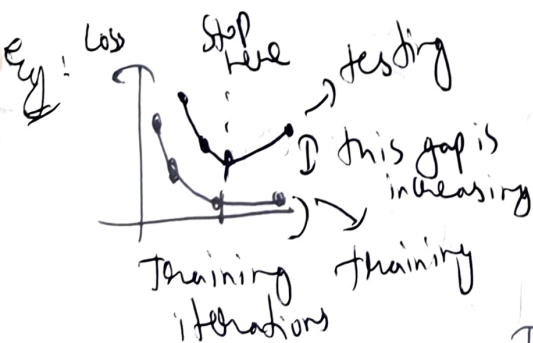
Underfitting $\rightarrow$ not trained properly

Overfitting $\rightarrow$ training very good, testing very bad

Regularization $\rightarrow$ techniques to overcome overfitting

e.g.: With some probability (say 50%) we set some activations to 0 Dropout

at one train case, some 50% are not functioning i.e. 0 and at another case other 50% are 0



Early Stopping

High Bias
High Variance

Low Bias
Low Variance

Low Bias
High Variance

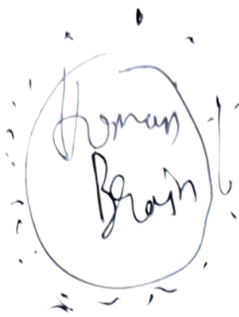
Underfitting: overfitting

Underfitting

Ideal case

overfitting

ANN



billions of neurons

helps in finding patterns around us

for eg. Baby seeing apple & learning about it

input $\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \Rightarrow$ output

Neurons



$f^{\wedge}$ with a output value, usually b/w 0 & 1

$$f(x) = 0.58$$

30x

30x

30x

→ just like how brain works, if we see something we like, some neurons gets activated & other neurons for other cases

Layer of Neurons used for

recognizing specific patterns

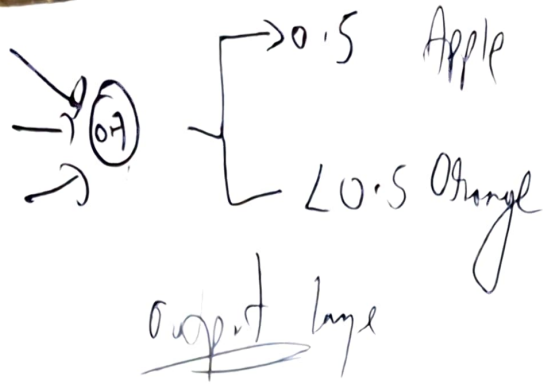
eg: red color, layer
orange color, layer

↳ also called Activation f's

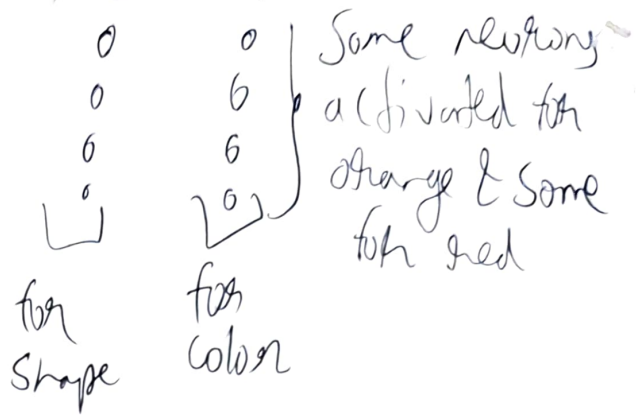
3 layers → input layer, hidden layer, output layer

(No. of neurons here = no. of features) rgb values

eg: Apple photo of 28px x 28px
no. of neurons = $28 \times 28 \times 3$

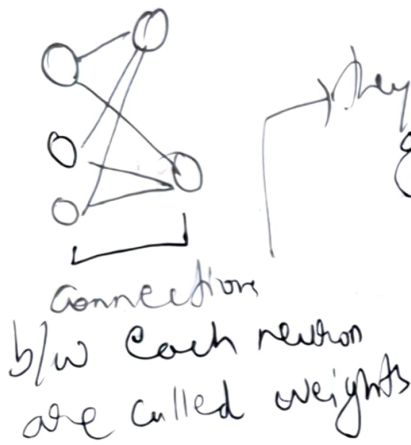


Hidden layers



1) Depending on our choice we can keep the no. of hidden layers/neurons

2) More neurons = More calculations = More time



They determine how much emphasis should be given to a certain region/pattern

$$\begin{matrix} a_{11} & \xrightarrow{w_{11}} & a_2 \\ & \xrightarrow{w_{12}} & \end{matrix}$$

$$w_{11} \times a_{11} + w_{12} \times a_{12}$$

weighted sum + b

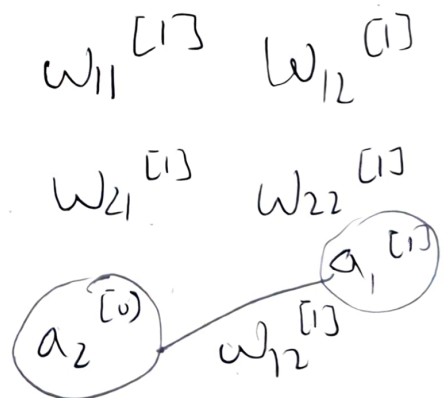
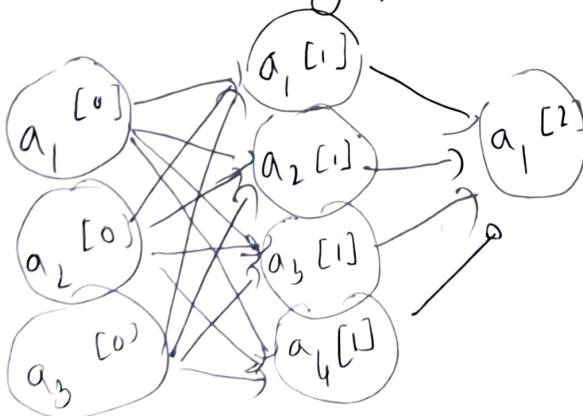
Activation $f \rightarrow$ Active/Inactive (start value) bias

$$a_{21} = f(w_{11} \times a_{11} + w_{12} \times a_{12} + b)$$

$$W = \begin{bmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \end{bmatrix}$$

These weights are at first set at random values & then after training, they will be adjusted accordingly

Notation



ReLU Hyper

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \\ w_{41} & w_{42} & w_{43} \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} \end{bmatrix}$$

neurons in hidden layer $(n-h, n-1)$ $(n-1, 1)$
 $(4, 3)$ $(3, 1)$ neurons in input layer

$$f\left(\begin{bmatrix} \end{bmatrix} + B1\right)$$

$$A_1 = f(W1 * A0 + B1) \quad A_2 = f(W2 * A1 + B2) = A1$$

Input $\rightarrow$ Hidden $\rightarrow$ Output] forward

Cost f^n (Error Represⁿ)

If the cost is slightly decreasing by small change in wt then, it is still not in Minimum cost, so decrease

According to the effect that weight gives

Cost f^n (high)

Basically small step in one direction (small +ve no.)

Learning rate

0 - 0 $\rightarrow$ 0 $\rightarrow$ X wrong

0 - 6 $\rightarrow$ 0 $\rightarrow$ X wrong

0 - 6 $\rightarrow$ 0 $\rightarrow$ X wrong



$$w = w - \alpha * \frac{\partial \text{Cost}}{\partial w}$$

$$B = B - \alpha * \frac{\partial \text{Cost}}{\partial B}$$

How much cost is affected by w

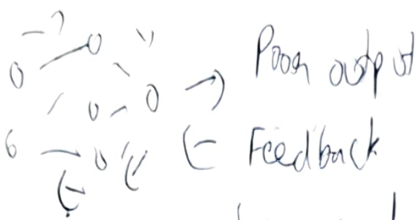
Slope

gradient descent algorithm to minimize loss/cost

All the weights & bias will use this and gets updated

For bettering cost f^n , the previous weights should be used

Back propagation



the good aspect

Forward
Propagation
Back

Propagation

For diffⁿ in pyrim → Micrograph / Subgraph

-) We don't have Active f^n 's, all the eqⁿ's will eventually follow same formulae i.e.

$$w' \times A0 + B'$$

→ This eqⁿ is linear, we need to utilize non-linear f's also if we have more than 1 hidden layer

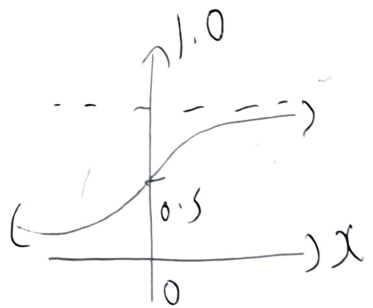
→ real world data

Types of Activation fⁿs

Sigmoid $f(x) = \frac{1}{1+e^{-x}}$

$$0 < f(x) < 1$$

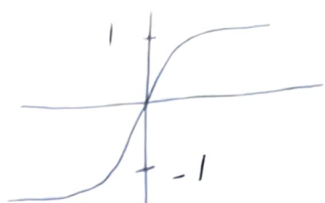
i.e. probability



-) Mostly used in output neuron in binary classification

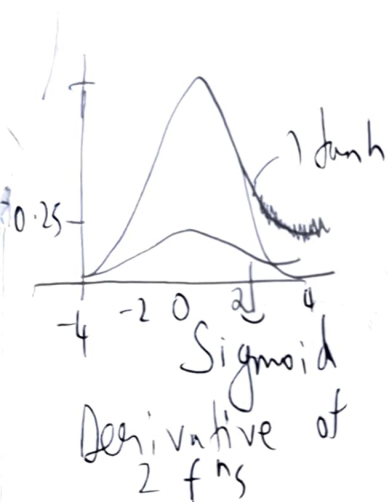
$$f'(x) = \frac{e^{-x}}{(1+e^{-x})^2} = \frac{f(x)}{(1-f(x))}$$

$$\tanh = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



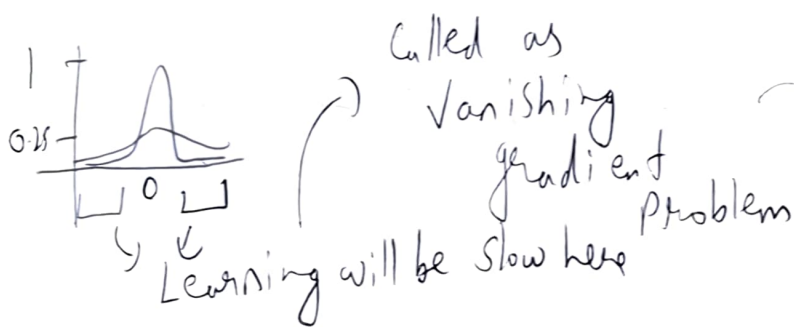
$$\tanh' = 1 - \tanh^2(x)$$

→ for $\frac{\partial \text{cost}}{\partial w}$, $\frac{\partial f^n}{\partial w}$ is also needed



→ here as the derivative of tanh is greater, $\frac{\partial \text{cost}}{\partial w}$ value can be greater, resulting in faster training in the hidden layers

→ Data is more centred towards 0, thus learning is faster

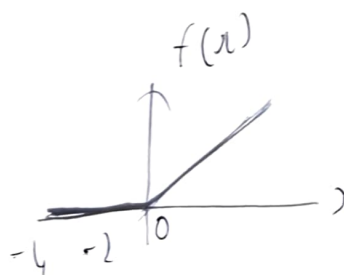


→ overcomed by ReLU

ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$

$$\frac{\partial f(x)}{\partial x} = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases}$$

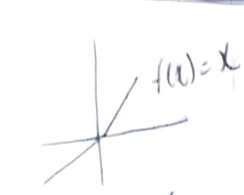


real world problems

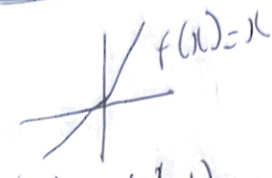
→ Not completely Linear

→ Has advantages of both Linear & non Linear property

Variations of ReLU


$$f(x) = 0,01 * x$$

Leaky Relu


$$f(x) = a(e^x - 1)$$

ELU
Exponential Linear
Unit

$$f(x) = \max(0, 0.01 * x_{ju})$$

$$f(x) = \max(a^x(e^x - 1), x)$$

Softmax function $f_i(x) = e^{a_i}$

$$e^T \cdot \begin{bmatrix} 7 \\ 0.5 \\ 1 \\ 3 \end{bmatrix} \rightarrow \begin{bmatrix} e^7 \\ e^{0.5} \\ e^1 \\ e^3 \end{bmatrix} \rightarrow$$

-) Multi class classification

$$\sum_k e^{a_k} \left[\begin{array}{c} e^{\frac{1}{A}} \\ e^{\frac{1}{A}} + e^{0.5} + e^{\frac{1}{A}} + e^{\frac{1}{A}} \\ \hline e^{0.5} \\ \hline A \\ \hline e^{\frac{1}{A}} \\ \hline A \\ \hline e^{\frac{1}{A}} \\ \hline A \end{array} \right] = \left[\begin{array}{c} 0.9782 \\ 0.00147 \\ 0.00242 \\ 0.0179 \end{array} \right]$$

- What if output is linear & can take any value?
Eg: House Price Prediction (0 - \$1M)

Ex: House Price Prediction (0 - \$1M)

(ii) No output f^n in output neuron is used

4) Linear Regression

Forn Regression $\rightarrow$ output is only no. $\rightarrow$ Loss $\rightarrow$ forn / observation

error = $|a_i - y_i|$ - Actual value

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n |a_i - y_i| \quad (\text{or}) \quad \frac{1}{2n} \sum_{i=1}^n |a_i - y_i|^2$$

$\downarrow$ $\downarrow$
 Mean Absolute Mean Squared
 Error Error

Binary Classification

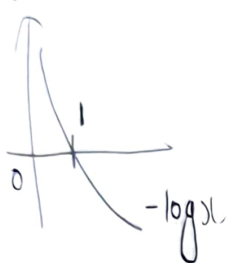
Binary Cross Entropy

$$\text{error} = - [y_i \log(a_i) + (1 - y_i) \log(1 - a_i)]$$

$$\text{cost} = -\frac{1}{m} \sum_{i=1}^m (\text{error})$$

Logistic Regression

known



if $y_i = 0$,

$$\text{error} = -\log(1 - a_i)$$

as a_i approaches 0, $1 - a_i$ approaches 1

=> less error

$a_i \rightarrow 0$ error $\downarrow$
 $a_i \rightarrow 1$ error $\uparrow$

as a_i approaches 1, $1 - a_i$ approaches 0

=> More error

if $y_i = 1$,

$$\text{error} = -\log(a_i)$$

$a_i \rightarrow 1$ error $\downarrow$
 $a_i \rightarrow 0$ error $\uparrow$

For Multi Class Classification

Categorical Cross Entropy $y_{i1} = 0$

y_i

$(0.1) \rightarrow$

$$y_i = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

$$a_j = \begin{bmatrix} 0.1 \\ 0.5 \\ 0.3 \\ 0.2 \end{bmatrix}$$

$(0.5) \rightarrow$

$$\text{error}_i = - [y_{i1} \log(a_{i1}) + y_{i2} \log(a_{i2}) + y_{i3} \log(a_{i3}) + y_{i4} \log(a_{i4})]$$

$(0.3) \rightarrow$

$(0.2) \rightarrow$

$$\text{cost} = -\frac{1}{m} \sum_{i=1}^m$$

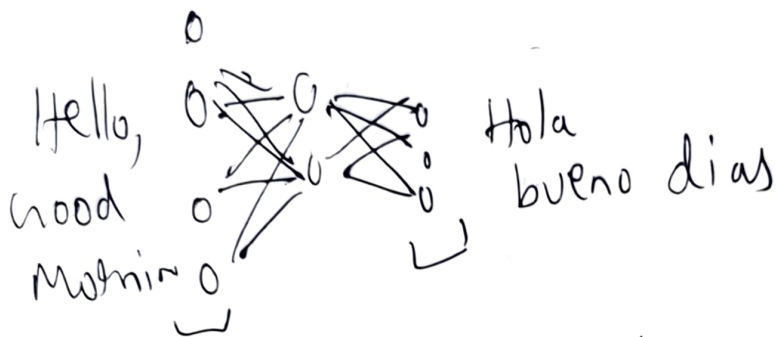
$$\text{cost} = -\sum_{j=0}^m \sum_{i=0}^m (y_{ij} * \log(a_{ij}))$$

RNN

→ auto complete, translate
virtual assistant

Why do we RNN?

→ In ANN, input & output layers are fixed



→ In ANN, arrangement doesn't matter

Kelly worked at Google → Kelly trabajó en Google

At Google, Kelly worked → En Google, Kelly trabajó

The sequence matters

For ANN seq. of 0
no. of 0
bedrooms

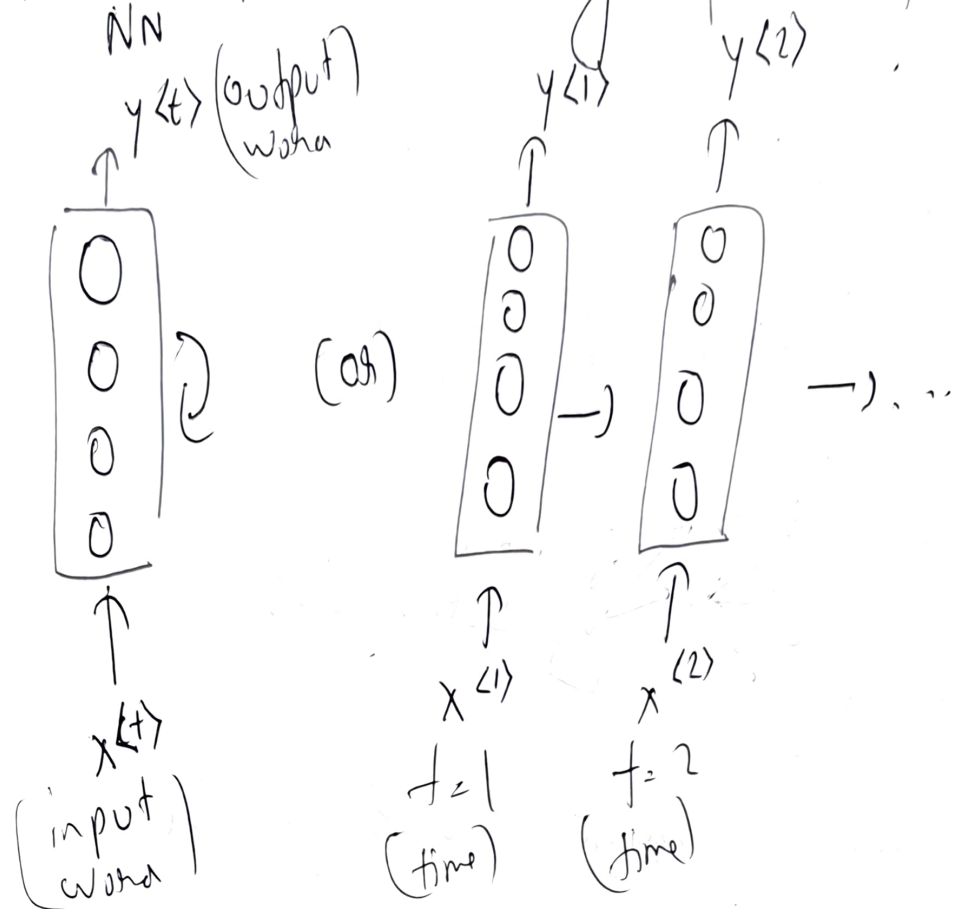
doesn't
matter

how the
data is
sequenced

Sequences → e.g.: DNA,
finance; voice, etc.

There are different types of
RNNs → GRU, Bi-directional
RNN

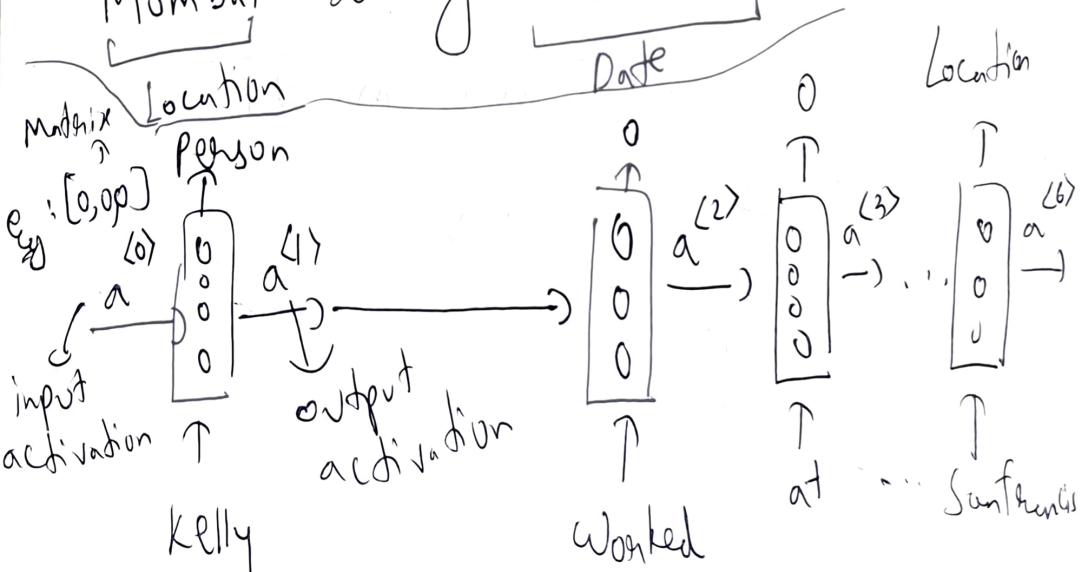
Recurrent $\rightarrow$ occurring repeatedly



eg: Named Entity Recognition

"Kelly is worked at Google in San Francisco"

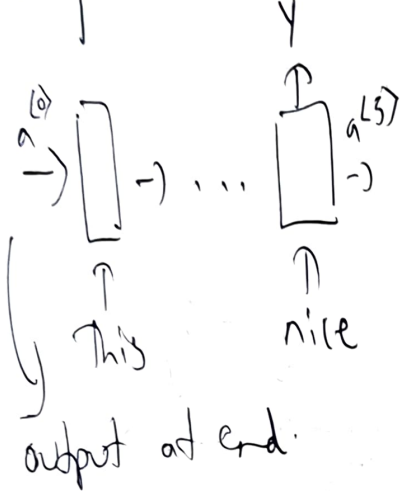
"Hey, show me best places to visit in Mumbai during Jan 2024"



Here output size = Input size

1) of RNN

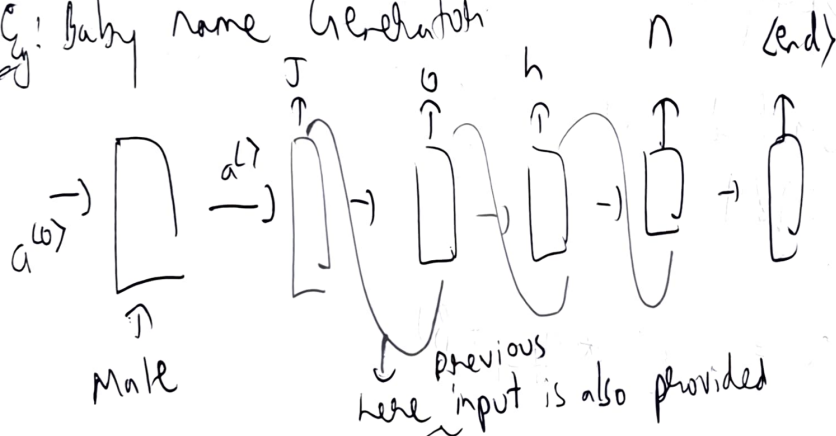
1) one to one RNN E_g : "This movie is nice"



based on this one number is decided $\rightarrow 5/5$

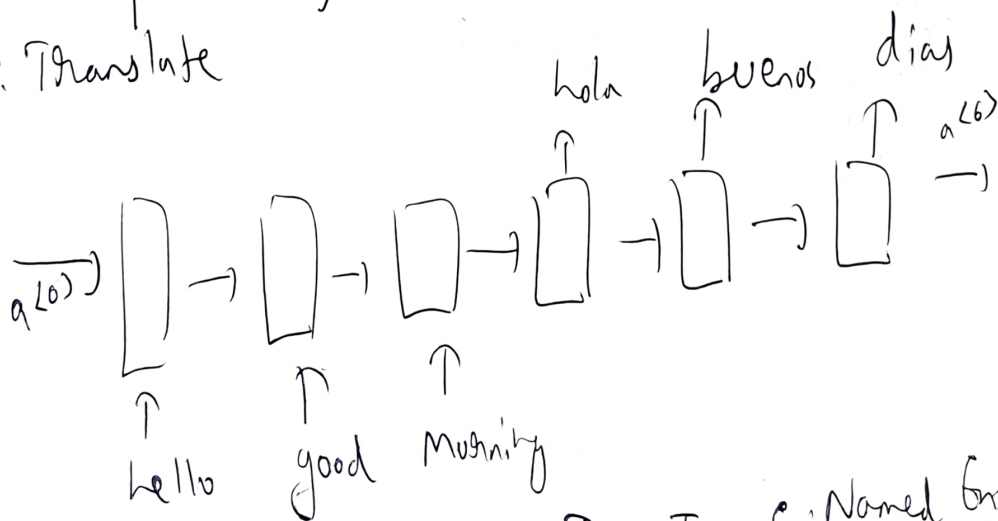
2) One to Many RNN

E_g : baby name generator



3) Many to Many RNN

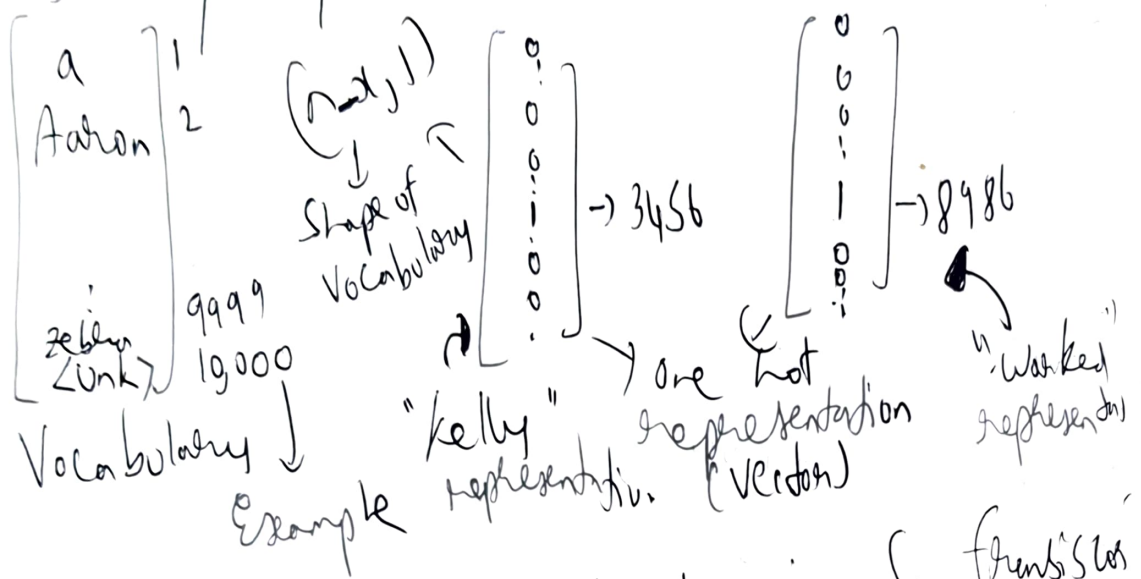
E_g : Translate



Many to many RNN $\rightarrow T_x = T_y$ E_g : Named Entity Recognition
 $\rightarrow T_x \neq T_y$ E_g : Translate

RNN Model Architecture

Computer doesn't understand words
() they only understand number / token



Example
kelly worked at Google in San Francisco
E: 3456 8989

$$\text{for } A^{(n)} \quad a = f(w * a_{-1} + b).$$

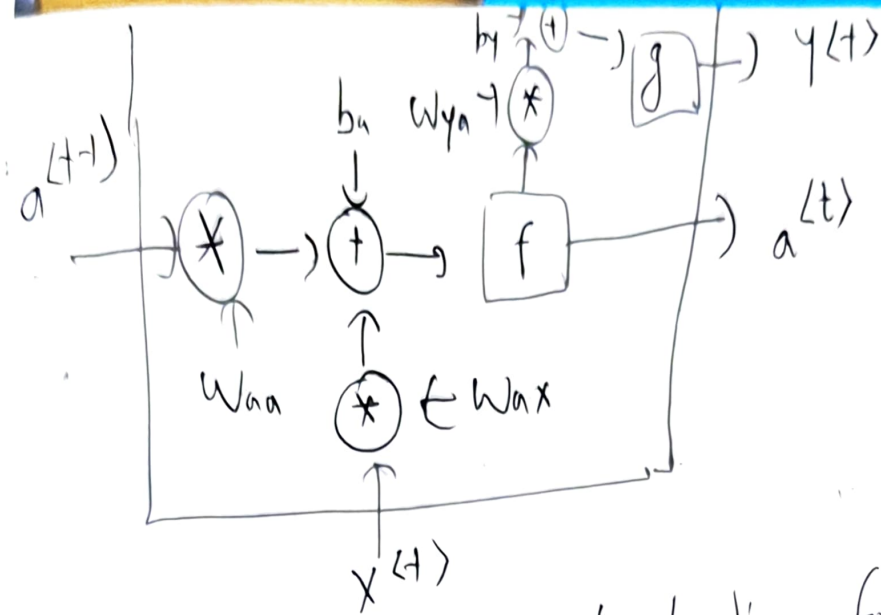
for RNN

2 in puts
are there

$a = f(w_{aa} * a_{t-1} + \underbrace{w_{ax} * x}_{\text{input } x} + b_a)$

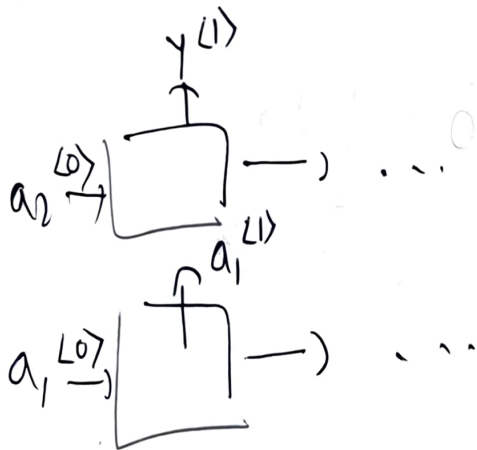
output
activation

$$\underset{\text{output}}{y} = \text{Softmax}(w_y a + b_y)$$



for many to one, no calculations for g at every turn but only at end.

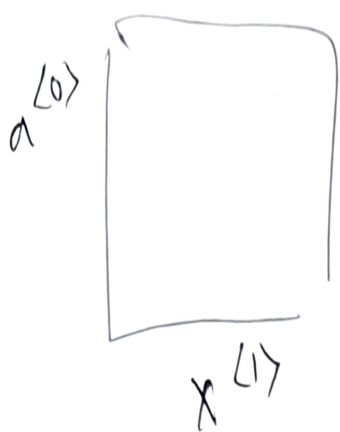
2 RNN cells,



Kelly

Back Propagation

eg: Movie Rating System



$$\text{Loss} = - \sum_{i=1}^n [y_i * \log(o_i)]$$

Here weight is only changed at the ending if after output is given

$$\frac{\partial L}{\partial w_{ya}} = \frac{\partial L}{\partial o} \times \frac{\partial o}{\partial w_{ya}} = (0 - y) \cdot dz$$

$\downarrow$ Prediction $\downarrow$ Actual Label

$$w_{ya} = w_{ya} - \alpha \frac{\partial L}{\partial w_{ya}}$$

$$\frac{\partial L}{\partial w_{aa}}, \text{ here } L \rightarrow 0 \rightarrow a_3 \rightarrow w_{aa}$$

$$L \rightarrow a_2 \rightarrow w_{aa}$$

$$L \rightarrow a_1$$

$$\downarrow$$

$$w_{aa}$$

$$\frac{\partial L}{\partial w_{aa}} = \frac{\partial L}{\partial o} \times \frac{\partial o}{\partial a_3} \times \frac{\partial a_3}{\partial w_{aa}} +$$

$$\frac{\partial L}{\partial o} \times \frac{\partial o}{\partial a_3} \times \frac{\partial a_3}{\partial a_2} \times \frac{\partial a_2}{\partial w_{aa}} + \frac{\partial L}{\partial o} \times \frac{\partial o}{\partial a_3} \times \frac{\partial a_3}{\partial a_1} \times \frac{\partial a_1}{\partial w_{aa}}$$

$$\frac{\partial L}{\partial w_{an}} = \sum_{i=1}^T \frac{\partial L}{\partial o} \times \frac{\partial o}{\partial a_i} \times \frac{\partial a_i}{\partial w_{an}}$$

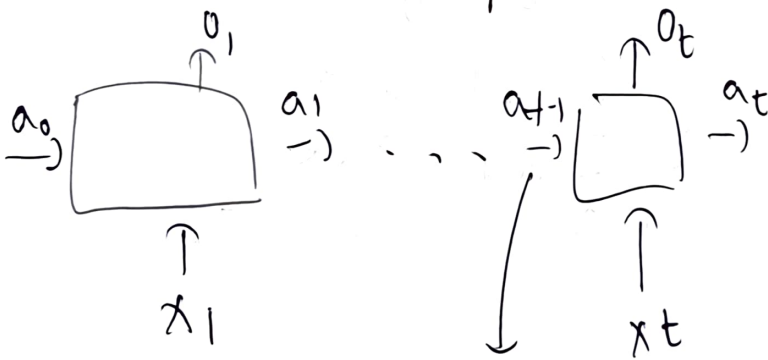
Input shape (batch, time, features)
(batch, 100, 4)

After Dense (256)
→ (batch, 100, 256)
RNN (16) timestep 1 → hidden state (16)

$\frac{\partial L}{\partial w_{a1}}$, also $L \rightarrow 0 \rightarrow a_3 \rightarrow w_{a1}$
 $\rightarrow a_2 \rightarrow w_{a1}$
 $\rightarrow a_1 \rightarrow w_{a1}$

LSTM why do we use LSTM over RNN?
 (Long Short term memory)

If a long sentence is given into RNN, it fails its operations



→ the value of only the previous words are only remember

Here the activation function loses the values from the starting part as there are so many operations being performed.

When we are back propagation

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial o} \times \left(\frac{\partial o}{\partial a_2} \times \frac{\partial a_2}{\partial a_{t-1}} \times \dots \right) \frac{\partial a_1}{\partial w}$$

$$\left(\frac{\partial J}{\partial a_1} \times \dots \right) \rightarrow 0 \times 0.2 \dots$$

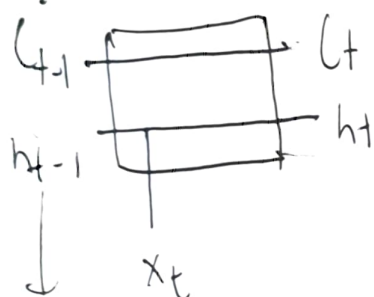
Very Small values continuing multiplying ≈ 0

This problem itself is known as Vanishing Gradient Problem

RNN $\rightarrow$ Short Term Memory only

LSTM $\rightarrow$ Long Term + Short Term

Memory cell state $\rightarrow$ will retain the data which is long term



hidden state

There are 3 gates used in LSTM,

1. forget gate
2. Input gate
3. Output gate

Gate : Allows certain information to pass through & restricts the other

$$c_t = (f_t * c_{t-1}) + (i_t * \tilde{c}_t) \rightarrow \text{i.e. same shape}$$

element wise multiplication

$$f_t \times (t-1)$$

$$\begin{bmatrix} 0 & 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & - \\ - & - \end{bmatrix}$$

eg: ↓

$$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0.1 & 0.2 & 0 & 0.3 \\ 1 & 0.3 & 0.2 & 0.1 \end{bmatrix}$$

$$\underline{0.1 \times 0} \quad \underline{0.2 \times 1}$$

Here when multiplied with 0, the information is forgotten (when with 1, it is retained)
element wise multiplication

1 → allow to pass

0 → discard

eg: Mr. Watson goes ... John wife...
This shall be remembered to identify this error

Here some details are defined

% of potential memory to remember $i_t \times \tilde{C}_t$ → Potential Long Term memory
Candidate Value
(potential new info. to be added)

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \leftarrow \text{Similar} \downarrow$$

$$a_t = \tanh(W_{aa} \times a_{t-1} + W_{ax} \times x + b_a)$$

or
 $\tanh(W_a [a_{t-1}, x_t] + b_a)$

$$[W_{aa} \quad W_{ax}] \times \begin{bmatrix} a_{t-1} \\ x_t \end{bmatrix}$$

Ex: Mr. Watson ... ^{this} her wife ~~was~~ angry.
 And tomorrow, he was need to go on a ~~date~~
 ↓
 will

Previous info + New info found

→ Not forgetting but adding new info while retaining the previous info

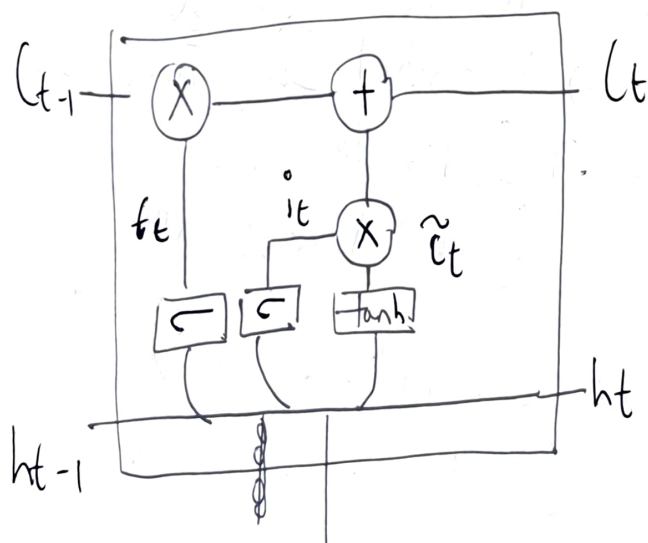
→ Sigmoid i.e. b/w (0 & 1)

$$f_t = \sigma (w_f \cdot [h_{t-1}, x_t] + b_f)$$

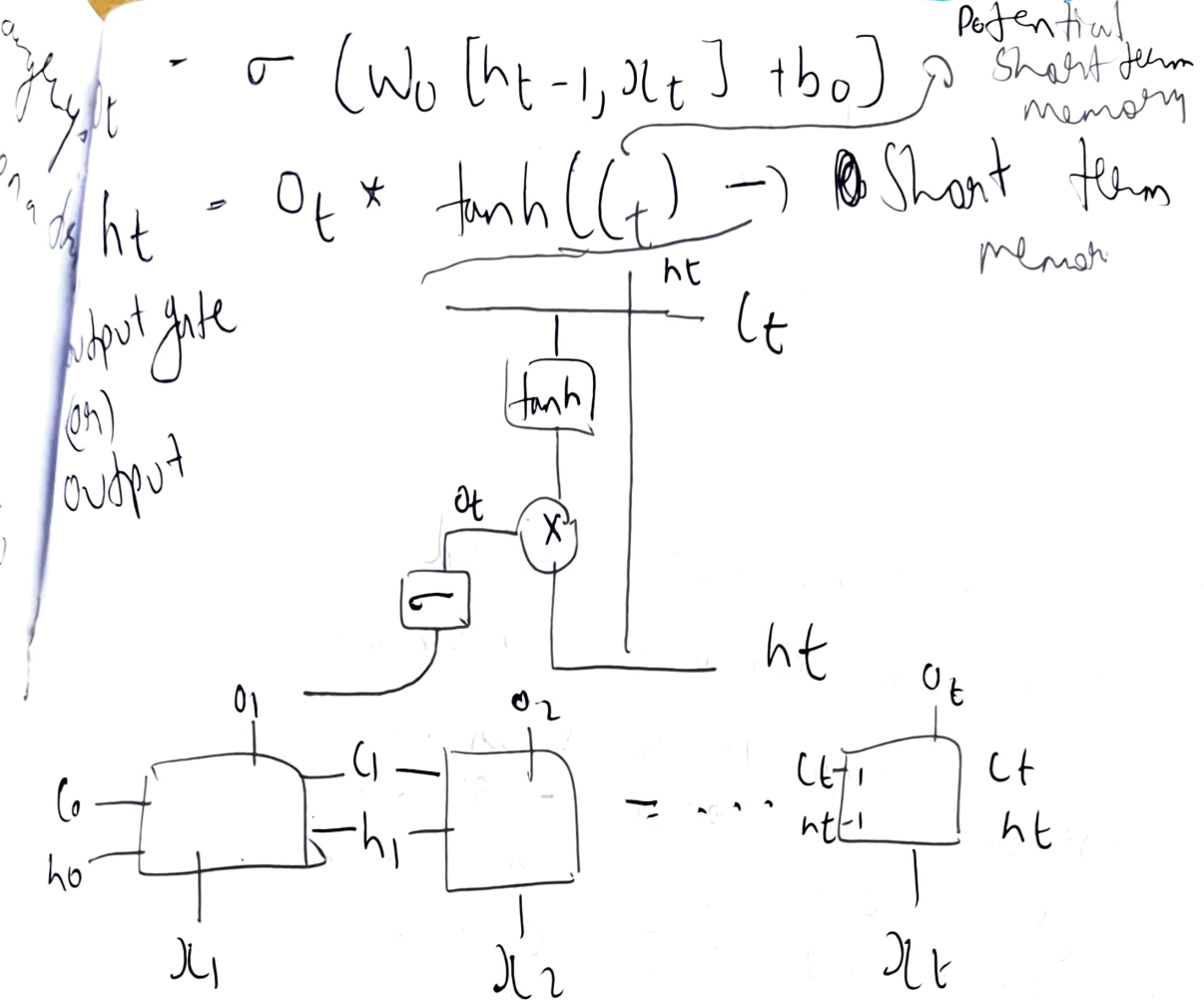
$$i_t = \sigma (w_i \cdot [h_{t-1}, x_t] + b_i)$$

when close to 0, forget info & when close to 1, retain that info

forget a little bit / remember a little bit

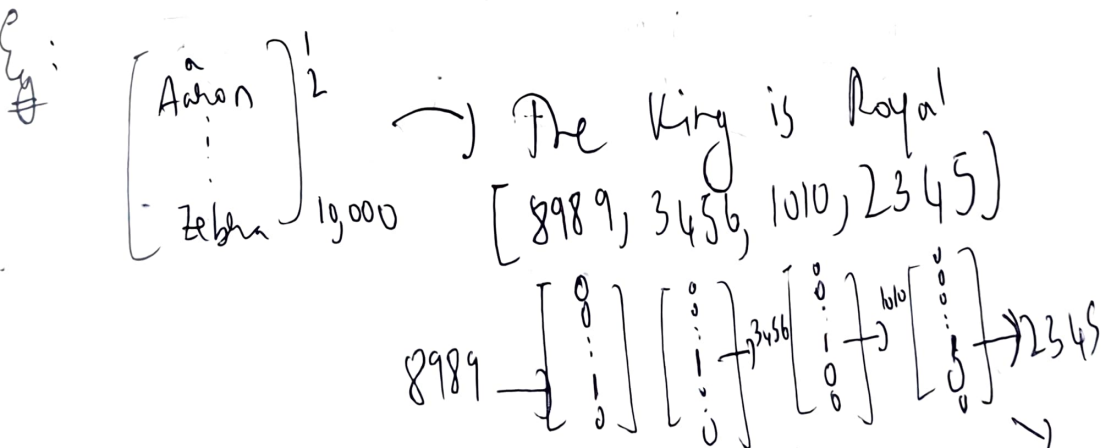


→ due to not being weights & bias in c_t , it doesn't vanish / explode



Computer doesn't understand words and to make them understand i.e. NLP, there are some techniques. Here we only discuss only a few

One-Hot Encoding



This has limitations as there will be no relation b/w words, for eg: apple (brown) orange $\rightarrow$ computer can't

oh, if we train for "He is King" the country, but our model, can't say any woman "She is a Prince of the country"

Also the matrix will be huge 10,000 or even bigger

as maybe this never came in the training data set

Featurized Representation - Word Embeddings

We train to get below matrix/vectors

cup lines semantic meaning interpretation

	King	Prince	Queen	...
Gender	0.99	0.99	-0.98	...
Age	0.75	0.4	0.2	...
Size	0.8	0.75	0.75	...
...	...	...	...	...

there may be 200 300

Here These are just example names and the model will understand this Word Embedding

Eg: Man, Woman : King, ? (doesn't capture meaning but "strong" meaning)

=)

King	-	Man	+	Woman	=	
0.99		0.1		-1		-1.01
0.75		0.2		0.6		0.65
...		...		...		

Each dimension have a meaning

Very close to green vector

How to Train

Word2Vec GloVe Pretrained models exist

eg: 12 Vec Google's Word2Vec $\rightarrow$ 300 dimensions

- After we first
- 1) make a dummy model & train
 - 2) Use the weights as the Vector Values

eg: 1) The future King is prince
2) :
10) :
} $\rightarrow$ Corpus of data
 $\downarrow$
collection of
written text
like novel, line,
para, etc...

$\rightarrow$ for our dummy model, we take 2 words

eg: future king
Source Target

$\rightarrow$ After removing stop words
future king prince
daughter princess
son prince
:
boy man

$\rightarrow$ doesn't add
meaningful info
eg: is, the, a, an,
etc...

$\rightarrow$ Now we create
Bigrams i.e. 2 continuous
words in a sentence.

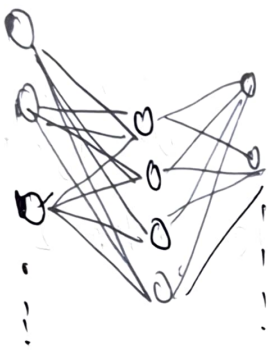
eg: Hi, my name is k.
Bigrams: ["Hi", "my"],
["my", "name"]
:

(future, king)
 (king, prince)
 :

now can vent for one hot representation

(boy, man)
 Input Output

[Beautiful]
 boy
 :
 king $\rightarrow$ less words

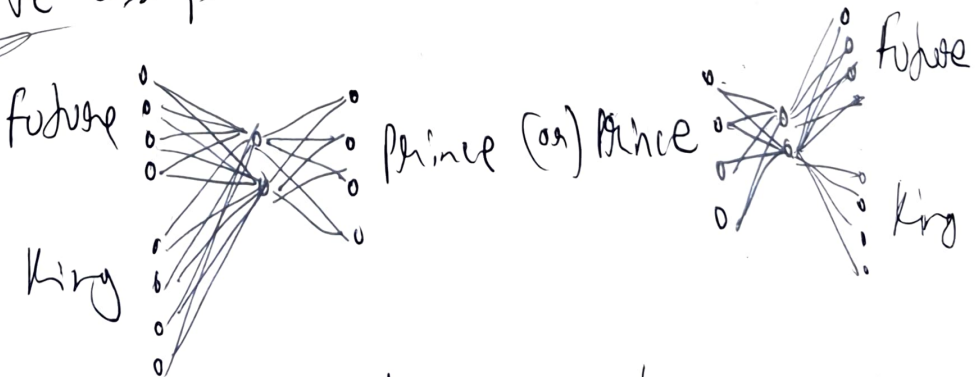


(Daughter, Princess)
 [" , Woman]
 [" , Beautiful]

- we do not care about the accuracy
 - we only care about weights, which will be adjusted to identify relation b/w daughter, princess ...

$[w_1, w_2, w_3, w_4] \rightarrow$ no. of hidden layer neurons

Alternative ways



2/3 words $\rightarrow$ 1 word
 Continuous Bag of Words (CBOW)

1 word $\rightarrow$ 2/3 words
 Skip gram